

SKILLFORGE

SkillForge - Build Guide

A step-by-step product, technology and execution roadmap

From problem -> MVP -> prototype -> demo -> scalable platform

Prepared from the conversation in this chat

Contents

- 01 Problem & opportunity**
- 02 Solution concept**
- 03 What to build in the MVP**
- 04 User journey
- 05 Industry simulation design
- 06 AI mentor design
- 07 Skill graph & scoring
- 08 System architecture
- 09 Database model
- 10 Technology stack
- 11 12-week implementation roadmap
- 12 Team roles
- 13 Security and quality checklist
- 14 Demo storyline
- 15 Learning resources
- 16 Future expansion

1. Problem & opportunity

The gap

Graduates can have strong theoretical knowledge and academic scores while still struggling with basic programming, debugging and practical application.

The opportunity is to create a measurable bridge between academic learning and actual junior-developer work. The product should make practical ability visible through observable evidence.

2. Solution concept

SkillForge is a practical employability platform built around realistic engineering workflows rather than isolated coding questions.

1	2	3	4	5	6
Learn	Solve	Debug	Build	Work	Prove
Micro concepts	Coding tasks	Broken code	Projects	Sprint simulation	Skill evidence

Product principle

Do not optimize for “How many questions did the student solve?” Optimize for “Can the student complete useful engineering work?”

3. What to build in the MVP

Module	Build now	Leave for later
Student auth	Login, profile	Advanced SSO
Assessment	Diagnostic test	Adaptive psychometrics
Code lab	Python editor + runner	Many-language matrix
Debug lab	Broken mini-apps	Large production repos
AI mentor	Hints + explanations	Autonomous agent workflows
Project lab	One end-to-end project	Project marketplace
Skill profile	Scores + evidence	External credentialing
Admin	Create tasks/tests	Full college analytics
Company	Basic candidate search	ATS integrations

4. User journey

Step	Student experience	Platform output
1. Diagnostic	30-45 minute assessment	Initial skill map
2. Roadmap	See strengths and gaps	Personalized sequence
3. Practice	Solve coding tasks	Correctness + skill signals
4. Debugging	Fix intentionally broken code	Debugging evidence
5. Project	Build from requirements	Portfolio evidence

6. Simulation	Complete sprint tasks	Work-style evidence
7. Profile	Review readiness score	Shareable skill profile

5. Industry simulation design

This is the signature feature. Each simulation should feel like a small, safe version of a real engineering ticket.

Template for a task

```
Task #142
Title: Fix login failure for expired sessions

Context:
Users intermittently receive HTTP 500 errors.

Acceptance criteria:
[ ] Expired sessions return a safe response
[ ] Valid sessions continue to work
[ ] Add/modify automated tests
[ ] Explain root cause

Evidence collected:
- Tests passed
- Time to resolution
- Commits / diff
- Hint usage
- Explanation quality
```

Simulation levels

Level	Example	Goal
L1 - Debug	Fix one broken function	Basic debugging
L2 - Feature	Add an API endpoint	Implementation
L3 - Integration	API + DB + tests	System thinking
L4 - Sprint	Multiple tickets	Prioritization + workflow
L5 - Team	PR + review + merge	Collaboration

6. AI mentor design

The AI should help a learner think, not remove the learning loop.

1	2	3	4
Hint	Clue	Concept	Review
Point to the area	Explain the likely cause	Teach the underlying idea	Explain the submitted solution

Guardrail

Automated tests should be the source of truth for correctness. The AI can explain failures, but it should not be the authority that marks a solution as passing.

7. Skill graph & scoring

Use a skill graph rather than a single “programming score”. A student can be excellent at syntax while weak at debugging, Git, SQL or testing.

Skill	Evidence examples	Weight idea
Programming	Correct solutions, code quality	20%
Problem solving	Medium/hard tasks	15%
Debugging	Bugs fixed, fix time	15%
Git	Branches, commits, PRs	10%
Database	SQL tasks, schema work	10%
APIs	Endpoint implementation	10%
Testing	Unit/integration tests	10%
Projects	Requirements completed	10%

Use weights as a starting point, not a final scientific formula. Calibrate them with real outcomes after pilots.

8. System architecture



For a first prototype, keep the architecture simple. The code runner is the subsystem that requires the most careful isolation and abuse controls.

9. Database model

```

users
-> student_profiles
-> company_profiles

skills

```

```

-> student_skill_scores

challenges
  -> test_cases
  -> submissions

projects
  -> project_tasks
  -> project_submissions

simulations
  -> simulation_tasks
  -> evidence_events

assessments
  -> assessment_questions
  -> assessment_attempts

```

10. Technology stack

Layer	Recommended	Why
Frontend	Next.js, React, Tailwind	Fast product development + dashboards
Editor	Monaco Editor	IDE-like browser experience
Backend	NestJS or FastAPI	Clean APIs and services
Database	PostgreSQL	Strong relational model
Auth	Clerk / Auth0 / Firebase Auth	Avoid building auth from scratch
Execution	Docker sandbox	Process isolation boundary
AI	LLM API	Mentor, review, recommendations
Design	Figma	Rapid UI iteration
Version control	Git + GitHub	Realistic engineering workflow
Deployment	Vercel + managed backend/db	Fast MVP deployment

Recommendation for a student team

Start with one language (Python) and one project. A small, polished vertical slice is much stronger than five unfinished subsystems.

11. 12-week implementation roadmap

Time	Phase	Key outcome
Week 1	Discovery + UX	Define personas, user journey, skill taxonomy, scoring rules, challenge template, and Figma screens.
Week 2	Foundation	Set up repo, Next.js frontend, backend, PostgreSQL, authentication, CI basics.
Weeks 3-4	Coding engine	Build Monaco editor, submission API, Docker runner, time/memory limits, test case evaluation.
Week 5	Learning layer	Challenge catalog, categories, difficulty, progress, skill events, dashboard.
Week 6	AI mentor	Hints, explanation flow, code review, and prompt/evaluation safeguards.
Weeks 7-8	Industry lab	First repository-style simulation, issue tracker, sprint tasks, PR-like submissions.
Week 9	Readiness engine	Evidence model, score calculation,

		learner profile.
Week 10	Company dashboard	Candidate search, filters, profile view, assessment assignment.
Week 11	Security + deployment	Sandbox hardening, abuse tests, logging, monitoring, deployment.
Week 12	Demo + polish	Seed data, UX cleanup, end-to-end rehearsal, pitch deck, backup demo path.

12. Team roles

Role	Main ownership	Key deliverables
Frontend	Student/company/admin UI	Dashboard, editor, profile
Backend	APIs + data model	Auth, challenges, scoring, APIs
Platform/Security	Runner + infrastructure	Docker execution, limits, deployment
AI/Assessment	Mentor + scoring	Prompts, hints, evaluation logic
Product/UX	User flows + content	Figma, task design, demo story

With a small team, people can own multiple areas. The important point is to assign clear ownership for the code runner, learner experience, and evidence model.

13. Security & quality checklist

- Never run untrusted student code directly on the application server.
- Use isolated execution with CPU, memory, time, process, storage, and network controls appropriate to your environment.
- Prevent unnecessary filesystem and secret access from student jobs.
- Rate-limit submissions and AI requests.
- Log execution results and important security events without storing unnecessary personal data.
- Keep automated tests deterministic; separate correctness from AI feedback.
- Build a simple fallback demo path in case external AI, hosting, or network services fail during presentation.

14. Demo storyline

1	2	3	4	5	6
Problem	Assess	Learn	Debug	Build	Hire
High marks, weak practice	Find real skill gaps	Target the gaps	Fix broken code	Ship a project	Company sees evidence

The strongest 3-5 minute demo

1) Show the problem. 2) Run a quick diagnostic. 3) Open a realistic debugging ticket. 4) Show the student fix and test result. 5) Open the readiness profile. 6) Switch to the company dashboard and find the candidate. Finish with the pitch: “Proof of Skill > Proof of Study.”

15. Learning resources

The links below are official or primary documentation resources that match the suggested stack. They are included as starting points for implementation and learning.

Next.js Documentation - <https://nextjs.org/docs> Frontend framework and application routing

React Documentation - <https://react.dev/learn> React fundamentals

Tailwind CSS Documentation - <https://tailwindcss.com/docs> Utility-first styling

Monaco Editor - <https://microsoft.github.io/monaco-editor/> Browser code editor

NestJS Documentation - <https://docs.nestjs.com/> Backend framework alternative

FastAPI Documentation - <https://fastapi.tiangolo.com/> Python backend alternative

PostgreSQL Documentation - <https://www.postgresql.org/docs/> Database

Docker Documentation - <https://docs.docker.com/> Containerization and isolation concepts

Git Documentation - <https://git-scm.com/doc> Version control

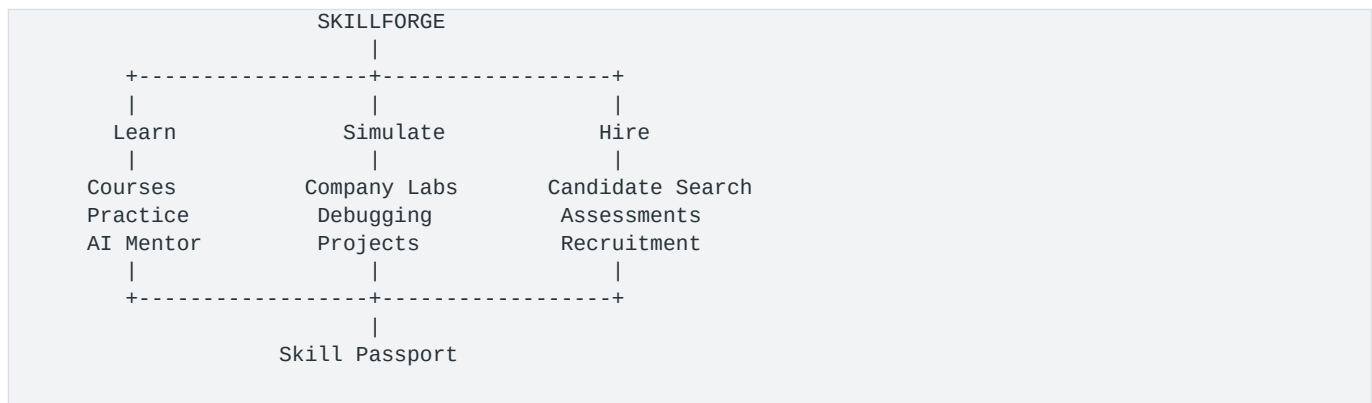
GitHub Docs - <https://docs.github.com/> Repositories, pull requests, issues

Figma Learn - <https://help.figma.com/hc/en-us/categories/360002051613> UI/UX design

Vercel Documentation - <https://vercel.com/docs> Frontend deployment

MDN Web Docs - <https://developer.mozilla.org/> Web fundamentals, APIs, JavaScript

16. Future expansion



Long-term, the platform can evolve into a portable “skill passport” based on practical evidence: tasks, simulations, projects, code reviews, and assessments. This becomes the bridge between education and hiring.

Final pitch

SkillForge bridges the gap between academic knowledge and industry readiness by letting graduates learn, solve, debug, build, and work in realistic software-engineering environments while generating verifiable evidence of their skills.

Appendix: first 7 days checklist

Day	Do this	Done when...
Day 1	Confirm product scope	One-page PRD is agreed
Day 2	Design user journey	Student flow works on paper/Figma
Day 3	Create monorepo + database	Local project runs
Day 4	Build auth + dashboard shell	A user can log in and reach dashboard
Day 5	Add Monaco editor	Editor loads and captures code
Day 6	Run one Python submission	One safe test case passes
Day 7	Create first debugging challenge	A complete task works end-to-end